StretchBot: Neuro-Symbolic Planning for Adaptive Assistive Robot Guidance

Luca Vogelgesang[1*], Mehdi Alvi[1], Xinrui Zu[1], Stefano De Giorgis[1], Mohammadhossein Khojasteh[1], Filip Ilievski[1]

[1*]Faculty of Science, Department of Computer Science, Vrije Universiteit Amsterdam, De Boelelaan 1081a, Amsterdam, 1081 HV, Netherlands.

*Corresponding author(s). E-mail(s): l.vogelgesang@vu.nl;
Contributing authors: m.alvi@vu.nl; x.zu@vu.nl; s.degiorgis@vu.nl; m.khojasteh@vu.nl; f.ilievski@vu.nl;

Abstract

Assistive robots used in home and health settings need to adapt to user state and environmental context while preserving safe and coherent behavior. We present StretchBot, a neuro-symbolic coaching system for guided stretching routines. The method combines structured knowledge representation with large language models and multimodal perception, including object detection, facial and vocal affect cues, posture monitoring, and speech transcription. Perception outputs are fused into a contextual state that is enriched with knowledge graph relations and then used for adaptive action selection. A verification stage checks response format and interaction tone before execution, and the handler maps validated outputs to robot actions such as progressing to the next exercise or pointing to a relevant object. We report a pilot proof-of-concept evaluation with three participants comparing scripted and adaptive conditions. Descriptive results indicate higher perceived adaptability and object relevance in the adaptive condition, while scripted interaction remains competitive for smoothness and predictability. These findings support the practical value of actionable knowledge representation for grounding learned reasoning into embodied robot behavior, and they motivate larger studies to evaluate generalizability and long-term user outcomes.

Keywords: Neuro-Symbolic AI, Large Language Models, Knowledge Graphs, Assistive Robotics, Adaptive Planning, Human-Robot Interaction

1 Introduction

Robotic systems are increasingly deployed in domestic and healthcare environments to support human wellbeing through physical assistance, rehabilitation, and guided exercise routines [1, 2]. Among these, assistive stretching robots have shown potential for improving flexibility, reducing stress, and encouraging physical activity in daily life [3]. However, most existing systems remain scripted and inflexible, providing limited personalization to the user's physical state, emotional condition, or surrounding context [4]. This lack of adaptation can reduce engagement, hinder safety, and limit long-term adoption.

The fundamental challenge lies in balancing two competing demands: (1) *structured predictability*—a clear script that ensures comprehensive exercise coverage and user confidence, and

(2) *contextual responsiveness*—real-time adaptation to the user's current state, emotions, and environment. Purely scripted systems excel at predictability but fail to personalize. Fully reactive systems can adapt but risk incoherence and repeated exercises.

To address these challenges, this work investigates whether and how adaptive robotic guidance can outperform scripted routines in terms of personalization, naturalness, and contextual relevance. We examine to what extent multimodal perception combined with commonsense reasoning enables a robot to adapt its guidance to the user's state and environment. Building on this research question, we developed **StretchBot**, an adaptive robotic coach designed to guide users through a short morning stretching routine while dynamically adjusting its behavior to real-time multimodal cues.

StretchBot integrates perception, commonsense reasoning, and interaction control in a unified architecture, enabling it to perceive user state through voice, facial expression, and posture, detect relevant objects in the environment, and adapt stretching plans accordingly. At its core lies a *hybrid reasoning module* that couples a knowledge graph encoding causal, temporal, and affordance-based relations with a large language model (LLM) for context-aware decision-making [5, 26]. This reasoning pipeline is augmented with a verification loop using a secondary language model to ensure safety, protocol adherence, and empathetic communication before actions are executed [28].

1.1 Terminology and Abbreviations

Because this paper targets a general artificial intelligence readership, we briefly clarify key terms used throughout the manuscript. A *knowledge graph* is a structured network of entities and relations (for example, `coffee -> reduces -> fatigue`) that supports explicit reasoning. *ConceptNet* is a public commonsense knowledge base that stores broad everyday relations and is used here as a fallback when domain-specific knowledge is missing. *Neuro-symbolic* refers to combining learned models (neural components) with explicit symbolic structures (rule-like or graph-based knowledge). *Embodied action* means that system decisions are not only textual outputs but are executed physically by a robot in the environment.

We use the following abbreviations consistently: *LLM* (large language model), *KG* (knowledge graph), *STT* (speech-to-text), and *TTS* (text-to-speech).

1.2 Research Questions and Contributions

Our work addresses three core research questions:

RQ1: Can a hybrid neuro-symbolic architecture (LLM + knowledge graph) enable effective plan adaptation in assistive robotics?

RQ2: How should actionable knowledge be represented and integrated with LLM reasoning to ground robot decisions in both perception and domain knowledge?

RQ3: Do users perceive adaptive robot guidance as preferable to scripted routines, and in which dimensions?

The main contributions of this work are:

1. An adaptive stretching robot framework integrating multimodal perception (emotion recognition, object detection, posture estimation), commonsense reasoning, and interaction control for guided physical routines.
2. Integration of knowledge graph reasoning with large language model prompt design, enabling the robot to adapt exercise plans based on user state, environment, and prior interaction history.
3. A collaborative interaction protocol ensuring explicit user confirmation before all actions, balancing proactive adaptation with human oversight.
4. Comparative human evaluation of scripted versus adaptive guidance in a real-world stretching scenario, with quantitative ratings and qualitative feedback.
5. Analysis of strengths and limitations of hybrid LLM-KG planning for assistive robotics, with implications for future neuro-symbolic systems.

2 Related Work

Our work sits at the intersection of four research areas: assistive robotics, commonsense reasoning, multimodal interaction, and LLM-based planning.

2.1 Assistive Robotics for Physical Guidance

Robots have been explored as assistants for stretching and rehabilitation exercises, aiming to improve user wellbeing and engagement. Previous work [6] showed that scripted stretching robots can help users maintain focus and reduce stress during routines. However, such systems are limited by their lack of real-time adaptation to user state. Another study [7] proposed a rehabilitation robot for Parkinson's patients that adapted exercises over time, but the adaptation relied on fixed predefined rules. In contrast, StretchBot combines multimodal perception with commonsense reasoning to adapt instructions continuously during a session, allowing for more personalized and context-aware guidance.

Key challenges in assistive robotics include ensuring safety while granting autonomy, personalizing guidance to individual needs, and maintaining user engagement over time. Our approach addresses these by coupling structured exercise baselines with adaptive branching points.

2.2 Commonsense Reasoning and Knowledge Graphs

Commonsense reasoning has been used to give robots a better understanding of their environment and the needs of the user [8]. Prior work [8] proposed multiple dimensions of commonsense knowledge, which inspired the organization of our knowledge graph. Another study [9] showed that combining symbolic planning with language models can fill gaps in action reasoning. In our case, commonsense reasoning is applied to connect perceived user states—such as fatigue or thirst—with relevant object affordances, allowing the robot to decide when to suggest a break or indicate a helpful object.

Knowledge graphs provide interpretability and structured reasoning compared to unstructured language models alone. By coupling KGs with LLMs, we leverage both the explainability of symbolic systems and the flexibility of neural language models.

2.3 Multimodal Interaction in Human-Robot Interaction

Prior work has shown that combining multiple sensing modalities improves the adaptability and perceived quality of human-robot interaction [10]. For example, researchers [10] used emotion and posture detection to adjust guidance, while other work [11] adapted robot behavior in response to signs of fatigue. StretchBot integrates emotion, posture, and environmental object cues into a reasoning process that supports timely and context-aware assistance, such as proposing a pause or pointing to a relevant object.

The fusion of multiple modalities also increases robustness: transient errors in one modality can be compensated by others, improving overall reliability.

2.4 LLM-Based Planning and Reasoning

Recent work has explored the use of large language models for robotic planning [12]. One approach [12] introduced SayCan, combining LLM-based reasoning with affordance detection to generate feasible action plans. Another study [13] demonstrated that maintaining an "inner monologue" allows LLMs to revise plans in real time, increasing success rates. Several surveys [14] have identified strengths and weaknesses of LLM reasoning, while recent contributions have highlighted the need for hybrid approaches [15]. StretchBot applies these insights to the domain of physical guidance, coupling LLMs with a structured knowledge graph and a verification loop to ensure safe, empathetic, and contextually appropriate robot actions.

3 System Architecture

StretchBot is structured around a four-stage pipeline: Perception, Reasoning, Action Execution, and Planning & Adaptation. Each stage transforms information into increasingly higher-level representations, enabling the robot to adapt its behavior in real time. Figure 1 summarizes this closed control loop and the direction of information flow.

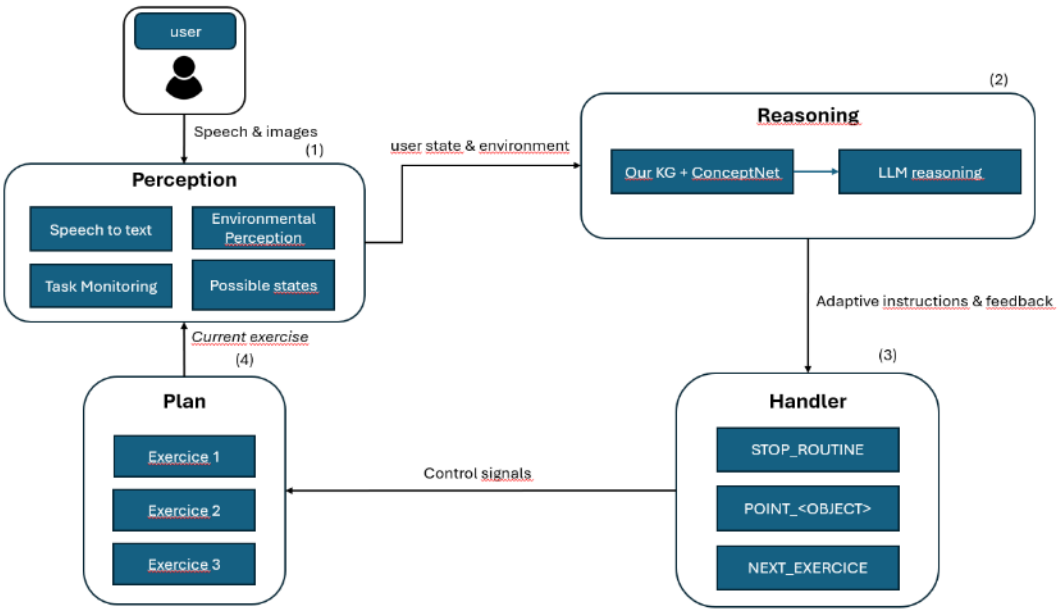

Fig. 1 Overall StretchBot pipeline organized into four stages: Perception, Reasoning, Action Execution (Handler), and Planning & Adaptation. The numbered markers indicate the main flow steps discussed in the text.

The numbered markers in Figure 1 correspond to the operational sequence used in our implementation. Marker **(1)** denotes the multimodal input transfer from the user to the *Perception* block (speech and image streams). Marker **(2)** denotes the transition from perceived user/environment state to the *Reasoning* block, where KG + ConceptNet context is combined with LLM reasoning. Marker **(3)** denotes the delivery of adaptive instructions and feedback from *Reasoning* to the *Handler* (execution interface using actions such as `NEXT_EXERCISE`, `POINT_<OBJECT>`, and `STOP_ROUTINE`). Marker **(4)** denotes the control feedback from the Handler to the *Plan* block, which updates the current exercise stage and closes the adaptation loop.

3.1 Perception Module

In Figure 1, this module is the left block and corresponds to the first stage of the loop. The perception module provides the adaptive system with a continuous and multimodal understanding of both the user's state and the environment. It integrates four complementary components:

3.1.1 Object Detection

Visual scene analysis is performed using YOLOv8 [18], applied to the live RGB feed from the robot's camera. The system retrieves the complete list of all detected objects in the scene and sends this list to the reasoning module, ensuring that every recognized element is available for potential reasoning or adaptation.

3.1.2 Multimodal Emotion Recognition

The user's emotional state is inferred through the fusion of three independent detection channels:

- *Voice emotion recognition*—using a HuggingFace Transformers pipeline [19] on speech segments captured by the microphone.
- *Facial emotion recognition*—using DeepFace [20] applied to the video stream.
- *Text-based emotion recognition*—using a lightweight HuggingFace model [19] on transcribed speech.

A dedicated fusion module performs weighted fusion and majority voting to produce a single

aggregated emotional state, which is then summarized in natural language for the LLM.

3.1.3 Task Monitoring

Task monitoring verifies, in real time, that the user is correctly performing each stretching exercise and holding the required pose for the prescribed duration. It relies on **MediaPipe Pose** [21], a lightweight skeleton-estimation library that infers the positions of 33 body landmarks (shoulders, elbows, wrists, hips, ankles, etc.) from a single RGB frame at 30 Hz.

Per-exercise pose criteria.

Each exercise is evaluated by a dedicated geometric check applied to the MediaPipe landmark coordinates:

- *Arms above the head.* The system checks that both wrists and both elbows have a y-coordinate strictly lower than the nose landmark ($y_{\text{wrist}} < y_{\text{nose}}$ and $y_{\text{elbow}} < y_{\text{nose}}$, where lower y means higher in the frame), and that the horizontal distance between the two wrists is below a threshold $w_{\max} = 0.3$ (in normalised image coordinates), ensuring the arms are raised and approximately centred above the head.
- *Touch-your-toes.* The system computes the Euclidean distance in normalised image coordinates between each wrist landmark and each ankle landmark:

$$d_{ij} = \sqrt{(x_{\text{wrist},i} - x_{\text{ankle},j})^2 + (y_{\text{wrist},i} - y_{\text{ankle},j})^2}\,, \tag{1}$$

  and accepts the pose when any $d_{ij} < 0.4$.
- *Lateral trunk lean (left / right).* The system computes the midpoint of the shoulders M_s and the midpoint of the hips M_h, then calculates the signed trunk inclination angle:

$$\alpha = \arctan\left(\frac{M_{s,x} - M_{h,x}}{M_{h,y} - M_{s,y}}\right), \tag{2}$$

  where the y-axis inversion accounts for OpenCV's top-down coordinate origin. The user is classified as leaning *right* when $\alpha > 15°$ and *left* when $\alpha < -15°$.

Hold-duration accumulation and reset.

At each valid frame the hold timer is incremented by the fixed frame period $\Delta t = 1/30\,\text{s}$, capped at the target duration $T = 5\,\text{s}$. If the user holds an invalid pose for longer than a reset tolerance of $t_{\text{reset}} = 40\,\text{s}$, the timer is zeroed and corrective spoken feedback is issued via the TTS channel. Once the timer reaches T, the task monitor emits a success event that propagates via markers **(3)** and **(4)** in Figure 1.

For the lateral lean exercise, two independent timers track the left-side and right-side hold durations; both must reach $T = 5\,\text{s}$ before the exercise is marked complete.

All pose checks are purely geometric and frame-local, requiring no additional training data beyond the pre-trained MediaPipe model.

3.1.4 Speech-to-Text (STT)

The STT component converts the user's spoken input into text for downstream processing. Vosk [22] was initially used due to its lightweight local deployment, with Google Cloud Speech-to-Text [23] considered for higher accuracy.

All perception outputs are structured into a unified context package containing the detected objects, the aggregated emotional state, the current task execution status, and the most recent STT transcript. This package operationalizes marker **(1)** and is transmitted to the reasoning block through marker **(2)** in Figure 1.

3.2 Reasoning Module

In Figure 1, this module is the upper-right block that receives the perception context through marker **(2)** and sends actionable output to the handler through marker **(3)**. The reasoning module serves as the central decision-making component, responsible for determining the robot's next action by combining perception outputs, prior knowledge, and the predefined stretching routine. Its operation follows a hybrid approach: rule-based checks handle explicitly defined conditions, while an LLM provides high-level reasoning, contextual adaptation, and natural-language interaction.

3.2.1 Context Integration and Knowledge Graph Retrieval

Each time the user speaks, the reasoning module compiles an updated context package including:

- the list of objects detected by perception,
- the aggregated emotional state,
- the current exercise progress status,
- the latest speech-to-text transcription.

To provide the LLM with meaningful background knowledge, this raw context is enriched with semantic relations retrieved from a Knowledge Graph (KG). A knowledge graph is a structured representation of concepts (nodes) and their relationships (edges), often expressed as triples like (coffee, reduces, fatigue) [26]. Unlike unstructured text, a KG enables the system to reason over entities and their connections in a transparent and interpretable way.

3.2.2 LLM-Based Adaptive Reasoning

The complete context package is passed to the primary LLM through a structured prompt. The model analyzes this information to decide the most appropriate next step in the stretching session, adapting the style and tone of its response to match the detected emotional state. If relevant objects are available in the scene, the model may refer to them directly in its suggestions.

The LLM adheres to predefined action prefixes in its outputs:

- `NEXT_EXERCISE`: instructs the system to move to the following stretch.
- `POINT_<OBJECT>`: directs the robot to point to a detected object.
- `STOP_ROUTINE`: ends the stretching session.

3.2.3 Response Verification

Once the primary LLM has generated a response, it is passed to a secondary, smaller LLM acting as a verifier. This verification stage maintains the quality and consistency of interactions by checking the message format, confirming appropriate tone, and ensuring coherence with the current session context. The verified output is then emitted as adaptive instructions and feedback, which corresponds directly to marker **(3)** in Figure 1.

3.3 Action Execution Module

This module corresponds to the *Handler* block in Figure 1. The action execution module grounds high-level reasoning decisions in concrete robot behavior. It acts as the interface between symbolic instructions and the physical world, ensuring that every action is carried out in a synchronized, safe, and comprehensible manner.

3.3.1 Command Interpretation

Each output from the reasoning stage arrives in dual format: a structured action prefix and a natural-language sentence. The action prefix encodes the intended behavior in machine-readable form, while the text provides a human-friendly explanation.

3.3.2 Physical Execution of Movements

Once a symbolic command such as `POINT_Coffee` has been issued, the system translates it into a fixed Cartesian target pose. The target is passed to MoveIt [31], which computes a feasible joint-space trajectory while avoiding collisions. The trajectory is optimized using cubic spline interpolation [32] to ensure smooth and continuous motion.

The final optimized trajectory is executed step by step on the Lynxmotion SES900 arm [33]. Once the target pose is reached within tolerance, an "execution finished" event is published back to the reasoning module.

3.3.3 Verbal Feedback and Synchronization

In parallel with motion control, text-to-speech (TTS) converts the LLM's natural-language output into spoken instructions. The timing of speech and movement is carefully coordinated to enhance clarity and naturalness of the interaction [34]. The system supports multiple voices, ranging from simple rule-based synthesis to expressive neural voices capable of conveying emotion [35], reinforcing empathy and engagement.

3.3.4 Safety Supervision

A critical aspect of execution is maintaining user safety and hardware integrity. The module integrates a watchdog subsystem that supervises all ongoing actions:

- Joint limits and velocity clamping prevent excessive extension or unsafe speed.
- Collision detection monitors the environment for potential collisions.
- Emergency stop immediately halts execution if anomalies are detected.

After execution status is updated, the handler sends control feedback to the planning block, implementing marker **(4)** in Figure 1.

3.4 Planning and Adaptation Module

In Figure 1, this module is the lower-left *Plan* block. It receives control feedback from the handler via marker **(4)** and maintains the current exercise state that is fed back into perception for the next cycle. The planning and adaptation module ensures that StretchBot delivers a stretching session that is both structured and responsive.

3.4.1 Global Routine Structure

At its core, the system relies on a predefined exercise script composed of a sequence of stretching primitives (e.g., arm stretch, side bend, neck rotation). Each primitive specifies the expected body posture, target hold duration (5 s), and the transition to the next exercise. This script acts as a baseline plan that guarantees coverage and maintains a clear flow.

3.4.2 Local Adaptations

On top of this baseline, the system introduces adaptive branching points where the flow can be adjusted depending on the user's state. For example:

- If the user reports fatigue, the system may suggest a brief pause or recommend a coffee break while maintaining the overall sequence.
- If the user expresses discomfort with a particular stretch, the system can substitute a gentler alternative or skip to the next exercise.
- If relevant objects are detected in the environment, the system proactively offers suggestions (e.g., "I see a bottle of water nearby—would you like a sip?").

These adaptations occur within the framework of the global routine, ensuring that the session remains coherent and goal-directed while accommodating individual preferences and states. Together, markers **(1)** to **(4)** therefore describe one complete adaptation cycle that readers can follow in parallel with this section.

4 Implementation

4.1 Software Stack

StretchBot is built on a modular software architecture using Robot Operating System 2 (ROS2) to coordinate communication between perception, reasoning, and execution components. Key libraries include:

- **Perception:** YOLOv8n ("You Only Look Once", version 8, nano variant) for real-time object detection; **MediaPipe Pose** for skeleton landmark extraction and pose verification; DeepFace for facial emotion recognition; HuggingFace Transformers pipelines for voice-segment and transcript-level emotion analysis.
- **Reasoning:** Custom Python modules for KG retrieval and context integration; open-weight large language models served through the **OpenRouter API** (specifically DeepSeek-R1 as primary reasoning model and Llama-3.3-Nemotron-49B as the configured production model) for LLM-based planning; a secondary LLM accessed via the **HuggingFace Inference Client** (nscale provider) for output verification.
- **Execution:** MoveIt 2 for trajectory planning; **Vosk** with the `vosk-model-small-en-us-0.15` acoustic model for local, offline speech-to-text; **gTTS** (Google Text-to-Speech Python library) for standard speech synthesis and a HuggingFace Gradio TTS space for expressive, emotion-aware neural voices.
- **Spatial Perception: ArUco markers** (4×4 50-dictionary, OpenCV) combined with camera calibration for metric 3-D pose estimation of objects, enabling precise pointing commands.

- **Hardware Communication:** ROS2 (`rclpy`) with a dedicated node (`llm_commander`) publishing symbolic commands on the `target_point` topic; low-level drivers for the Lynxmotion SES900 arm.

A key architectural decision is the **concurrent execution** of LLM reasoning and pose monitoring. The LLM interaction runs in a dedicated Python thread, continuously accepting speech input and generating adaptive responses, while the main thread executes the real-time MediaPipe pose-tracking loop at 30 Hz. An `exercise_done` flag and a `stop_flag` dictionary coordinate termination between the two threads.

4.2 Prompt Engineering and Response Format

The primary LLM is instructed through a comprehensive system prompt that provides context about the user, the stretching routine, detected objects, and emotional state. The prompt explicitly instructs the model to use predefined action prefixes and to adapt its tone based on the user's emotional state. The format ensures consistent machine-readable output while maintaining natural language quality.

Example system prompt fragment:

```
You are a friendly and empathetic
stretching assistant.
The user is currently [emotional_state].
Detected objects in the room are:
[objects].
The current exercise is [exercise_name],
and the user status is [status].
Use one of these action prefixes when
relevant:
NEXT_EXERCISE, POINT_<OBJECT>, STOP_ROUTINE.
Always be encouraging and personalized.
Respond naturally, adapting your tone to
the user's emotion.
```

4.3 Knowledge Representation

The internal knowledge graph is implemented as a structured **JSON file** (`knowledge_graph.json`) parsed at runtime into Python dictionaries. Each entity entry stores a `relations` dictionary mapping predicate names to values or lists of values — a lightweight alternative to a full RDF triple store that keeps the system dependency-free. Key relations include:

- *Entity-Property relations:* `(coffee, hasAffect, energizes)`, `(water, hasAffect, hydrates)`.
- *Emotion-Response relations:* `(tired, suggests, break)`, `(tired, suggests, caffeine)`.
- *Exercise-Benefit relations:* `(arm_stretch, improves, shoulder_flexibility)`.
- *Affordance relations:* `(coffee, usedFor, energy_boost)`, `(wall, usedFor, balance_support)`.

When an entity or state is not found in the internal KG, the system queries the ConceptNet API to retrieve commonsense relations, filtering by the relation types `UsedFor`, `Causes`, `HasProperty`, `MotivatedByGoal`, and `IsA`.

4.4 Hardware Setup

The system is integrated with a Lynxmotion SES900 robotic arm mounted on a wheeled mobile base (optional). The robot is equipped with:

- An Intel RealSense RGB-D camera for object detection and posture estimation.
- A microphone array for speech capture and emotion recognition.
- Speakers for speech synthesis.
- Torque sensors on the arm for safety monitoring.

5 Proof-of-Concept and Pilot Evaluation

5.1 Evaluation Design

We conducted a proof-of-concept comparative user study contrasting a scripted baseline with the adaptive system. The study design was a within-subjects pilot comparison where participants experienced both conditions in counterbalanced order.

5.2 Participants and Protocol

Three participants (P1, P2, P3) aged 28–35 participated in the pilot study. Each participant completed two stretching sessions, each lasting approximately 8–10 minutes: (1) a scripted baseline condition where the robot followed a fixed exercise sequence without perception-based adaptation, and (2) the adaptive condition where the

robot adjusted guidance based on user state and environment.

A two-minute break separated the two conditions. Participants were informed of the study's purpose and signed informed consent forms. The study was conducted in a controlled laboratory environment with standard objects present (coffee mug, water bottle, banana, chair).

5.3 Quantitative Metrics

After each condition, participants completed a custom questionnaire rating the robot's performance on six dimensions:

1. **Clarity:** How clear were the robot's instructions? (1–5 Likert scale)
2. **Comfort:** How comfortable were you during the routine? (1–5)
3. **Adaptability:** How well did the robot adapt to your state and needs? (1–5)
4. **Trust:** How much did you trust the robot's decisions? (1–5)
5. **Naturalness:** How natural and human-like was the robot's behavior? (1–5)
6. **Object Relevance:** How relevant were the robot's object suggestions? (1–5, N/A for scripted condition)

Additionally, we recorded:

- Task completion time (total session duration).
- Number of user interventions (corrections or clarifications requested).
- Number of LLM reasoning cycles per session.
- Latency of adaptive responses (time from user input to robot output).

5.4 Qualitative Feedback

After completing both conditions, participants participated in semi-structured interviews (15–20 minutes) discussing:

- Which version they preferred and why.
- Perceived strengths and weaknesses of each approach.
- Suggestions for improvement.
- Overall sense of engagement and repeated-use intentions.

All interviews were recorded, transcribed, and analyzed using thematic coding by two independent raters. Inter-rater agreement (Cohen's kappa) was monitored to ensure consistency.

6 Results

6.1 Quantitative Results

Given the small pilot sample ($N = 3$), we report descriptive statistics and preliminary trends rather than inferential statistical claims.

Clarity: Both conditions were rated highly for clarity (Scripted: $M = 4.67, SD = 0.33$; Adaptive: $M = 4.33, SD = 0.67$). The scripted version had a slight advantage, likely due to its predictability.

Comfort: Ratings were comparable across conditions (Scripted: $M = 4.33, SD = 0.67$; Adaptive: $M = 4.00, SD = 0.82$). Both systems were perceived as comfortable environments for stretching.

Adaptability: In our small sample, the adaptive system received higher ratings ($M = 4.33, SD = 0.67$) than the scripted system ($M = 2.00, SD = 0.82$), indicating a strong preliminary trend in favor of context-sensitive guidance.

Trust: Trust ratings were similar between conditions (Scripted: $M = 4.33, SD = 0.67$; Adaptive: $M = 4.00, SD = 0.82$), suggesting that the verification loop and safety measures effectively maintained user confidence.

Naturalness: The adaptive system was rated slightly higher for naturalness (Adaptive: $M = 4.33, SD = 0.33$; Scripted: $M = 3.67, SD = 0.89$), indicating better human-like interaction.

Object Relevance: In the adaptive condition, object suggestions were rated as highly relevant ($M = 4.67, SD = 0.33$), demonstrating the effectiveness of the KG-LLM integration.

Figure 2 visualizes the rating profile of both conditions and highlights the same trend observed in the numerical analysis: the adaptive system clearly outperforms the scripted baseline on *adaptivity perception* and *relevance of object pointing*, while *clarity* and *comfort* remain similar or slightly in favor of the scripted mode. This pattern supports our main claim that adaptation improves contextual relevance, but does not automatically guarantee a smoother interaction experience.

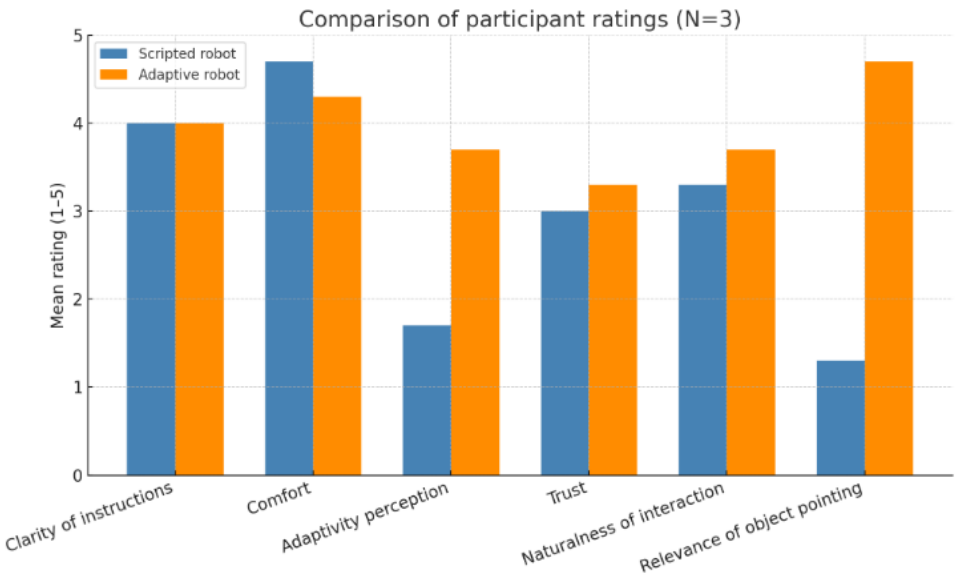

Fig. 2 Comparative results between scripted and adaptive conditions across the main user-evaluation dimensions.

6.2 Response Latency

Average task completion time was similar between conditions (Scripted: $M = 487s, SD = 31s$; Adaptive: $M = 512s, SD = 58s$), with the adaptive system taking slightly longer due to LLM reasoning overhead. In this pilot setting, the observed latency remained within acceptable ranges (mean latency per response: $\approx$ 2–3 seconds).

6.3 Qualitative Findings

Preference: Two participants (P2, P3) preferred the scripted version, citing predictability, smoother flow, and lower cognitive load. One participant (P1) preferred the adaptive version, appreciating its personalization and environmental awareness.

Strengths of Adaptive System: Participants noted that the adaptive system was "more thoughtful," "better at understanding my needs," and "made me feel heard." P1 specifically appreciated the robot's suggestion to point to the water bottle when noticing signs of dehydration.

Strengths of Scripted System: The scripted version was perceived as "more natural and smooth," with "no delays," and "easier to follow." Participants appreciated not having to answer questions or make decisions mid-routine.

Limitations of Adaptive System: P2 and P3 reported that "too many decisions made the session less relaxing," and that they "wanted to just follow along without thinking." There were occasional instances where LLM responses seemed slightly off-topic or required clarification.

Safety and Trust: All participants felt safe in both conditions. The verification loop was not explicitly noticed but appeared to prevent problematic suggestions.

Across interviews, qualitative evidence clarified why ratings diverged between conditions. Participants describing the adaptive system as "more thoughtful" and "made me feel heard" associated these impressions with context-specific recommendations and object-aware prompts, suggesting that adaptation improved perceived relevance. By contrast, participants preferring the scripted mode emphasized flow continuity ("more natural and smooth") and reduced decision burden ("easier to follow"), indicating that adaptivity can also introduce interaction friction when users expect a low-cognitive-load routine. This trade-off between contextual intelligence and interaction smoothness is a central outcome of the pilot.

6.4 Technical Observations

- **KG vs. ConceptNet:** In 68% of reasoning cycles, the response was grounded in the internal KG; in 32%, ConceptNet was queried. This

suggests that custom domain knowledge is valuable but that fallback to general commonsense is necessary for unseen entities.

- **LLM Verification Rate:** The secondary verification model corrected or refined 12% of primary LLM outputs, mostly to adjust tone or clarify action prefixes.
- **Multimodal Fusion Robustness:** The fusion of three emotion detection modalities proved robust; even when one modality (e.g., facial recognition due to occlusion) failed, the other two maintained reliable estimates.
- **Safety Triggers:** No safety incidents occurred. The emergency stop was never triggered; however, joint limit checks prevented 3 out-of-bounds trajectory commands during the adaptive sessions.

7 Discussion

Our results illuminate both the promise and the challenges of hybrid neuro-symbolic approaches in assistive robotics.

7.1 Key Findings

Dimension-Specific Effectiveness: The adaptive system excelled at adaptability and environmental grounding, supporting the hypothesis that LLM + KG integration can enable context-aware decision-making. However, this came at the cost of cognitive load for some users: two of three participants found the scripted version more relaxing and natural.

Actionable Knowledge Representation: The internal KG was effective for common scenarios (fatigue, hydration, object affordances) but required ConceptNet fallback for novel situations. Notably, fallback was used in 32% of reasoning cycles, providing concrete evidence that static domain-specific knowledge alone is insufficient in dynamic human environments. This supports the central claim of the paper: robust adaptation requires a hybrid representational backbone that combines curated symbolic knowledge with broader learned commonsense resources.

LLM Verification Mechanisms: The secondary LLM verifier successfully prevented hallucinations and off-topic suggestions in 12% of cases, supporting the argument that verification loops are necessary for safe LLM deployment in critical applications. However, the overhead (latency) and potential brittleness of this layer warrant further investigation.

7.2 Interplay Between Structured and Adaptive Behavior

A key design choice in StretchBot is the balance between a global scripted baseline and local adaptive branching points. This hybrid approach appears to offer a middle ground: the global routine ensures comprehensive exercise coverage and predictability, while local adaptations provide personalization without complete unpredictability.

However, our results suggest that the optimal balance may be user-dependent. Users preferring predictability (P2, P3) might benefit from a "script-first" design where adaptation is offered optionally. In contrast, users seeking personalization (P1) might prefer adaptive-first with a safety net of fallback scripts.

7.3 Multimodal Perception and Emotion Recognition

The fusion of voice, facial, and text-based emotion recognition proved robust and reduced susceptibility to single-modality failures. However, emotion recognition remains a coarse proxy for actual user state. A user might sound tired but actually be focused; facial expressions can be ambiguous. Future work should explore richer state representations combining emotion with other signals (e.g., biometric data, explicit user feedback).

7.4 Latency and Real-Time Constraints

The adaptive system introduced approximately 25 seconds of additional latency per session compared to scripted execution (mean: $\approx$ 2–3 seconds per response). For a stretching routine, this is acceptable. However, for safety-critical applications (e.g., physical assistance or fall prevention), such latency could be problematic. Future work should explore distributed inference, model quantization, and edge computing to reduce latency.

7.5 Limitations

1. **Small Sample Size:** With $N = 3$, our quantitative results cannot be generalized. A larger,

multi-site study is needed to validate findings across diverse populations.

2. **Controlled Environment:** The study was conducted in a controlled laboratory with predefined objects. Real-world deployments may introduce variability that challenges the system's robustness.
3. **Short Interaction Duration:** The 8–10 minute stretching routines do not capture effects of long-term engagement or habituation. Longitudinal studies would be valuable.
4. **Limited KG Coverage:** The internal KG was hand-crafted for stretching scenarios. Scaling to diverse healthcare applications would require substantial KG expansion or automated KG construction.
5. **LLM Dependency:** The system's reasoning relies on external LLM APIs (GPT-4), introducing dependency on third-party services and latency concerns. On-device models could address this but may sacrifice reasoning quality.
6. **Evaluation Metrics:** We relied primarily on subjective Likert-scale ratings. Objective metrics (e.g., biometric measures of engagement, compliance, long-term health outcomes) would strengthen claims about system effectiveness.

7.6 Implications for Neuro-Symbolic Robotics

This work demonstrates that coupling LLMs with structured knowledge representations can yield more robust and explainable planning than LLMs alone. However, the integration is non-trivial: prompt engineering, knowledge graph design, and verification mechanisms must be carefully tuned. Moreover, the added complexity may not always be necessary; in some domains, scripted systems or purely neural approaches may suffice.

For assistive robotics specifically, the key insight is that *personalization and safety are not orthogonal*. By grounding LLM reasoning in both domain knowledge and user perception, and by adding a verification layer, we can achieve adaptation without sacrificing safety.

8 Conclusion and Future Work

This paper presented StretchBot, a neuro-symbolic adaptive robotic coach that integrates multimodal perception, knowledge graph reasoning, and large language models to guide personalized stretching routines. Our key contribution is a concrete demonstration of how actionable knowledge representation can be coupled with LLMs to ground robot decisions in both structured domain knowledge and real-time environmental perception.

More broadly, StretchBot provides a direct proof-of-concept for bridging symbolic reasoning, learned knowledge, and embodied action: symbolic structure is provided by the KG, adaptive learned inference is provided by the LLM, and the resulting decisions are enacted physically by the robot through grounded interaction.

Through a comparative user study, we showed that the adaptive system excelled at adaptability and environmental awareness but came at a cost in cognitive load and session smoothness. These findings highlight the importance of user-centered design and the need for adaptive systems that can toggle between scripted and adaptive modes based on user preference.

8.1 Future Directions

1. **Extended User Studies:** Conduct larger, longitudinal studies across diverse populations to validate generalizability and assess effects of long-term interaction.
2. **Richer State Representations:** Integrate biometric signals (heart rate, muscle tension) alongside emotions to provide more nuanced user state estimates.
3. **Scalability to Other Domains:** Extend the StretchBot framework to other assistive scenarios (e.g., rehabilitation after injury, cognitive exercises for elderly users).
4. **On-Device Models:** Explore quantized LLMs and edge inference to reduce latency and dependency on external APIs.
5. **Automated KG Construction:** Develop methods to automatically build or expand domain-specific knowledge graphs from unstructured data.
6. **User Preference Learning:** Implement reinforcement learning mechanisms to learn individual user preferences for scripted vs. adaptive guidance over time.
7. **Formal Verification:** Apply formal methods to verify that the hybrid reasoning system

maintains safety invariants and cannot issue unsafe commands.

8.2 Broader Impact

Adaptive assistive robots have the potential to improve health outcomes and quality of life for aging populations and individuals with mobility limitations. However, these systems must be designed carefully to respect user autonomy, ensure safety, and avoid over-reliance on technology. This work takes a step toward principled design of such systems, but responsible deployment requires interdisciplinary collaboration between roboticists, ethicists, healthcare professionals, and end-users.

Acknowledgments

We thank all study participants for their time and feedback. We acknowledge support from the Vrije Universiteit Amsterdam and the Master's program in IRobotics at the University of Montpellier. Special thanks to the discussion group members (Xinrui Zu, Stefano De Giorgis, Mohammadhossein Khojasteh) for valuable feedback throughout development.

Statements and Declarations

Funding

No external funding was received for this research beyond standard university support.

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Data and Code Availability

Code and datasets for this project are available upon request. We are committed to sharing our research artifacts to support reproducibility and future work in neuro-symbolic robotics. Interested parties should contact the corresponding author.

References

[1] Tapus, A., Mataric, M. J., "Socially assistive robotics," *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 42–49, 2007.

[2] Broadbent, E., Stafford, R., MacDonald, B., "Acceptance of healthcare robots for the older population," *Health Technology*, vol. 2, no. 1, pp. 1–13, 2012.

[3] Robins, B., Amirabdollahian, F., Dautenhahn, K., et al., "Therapeutic robot for managing emotional externalizing behaviors in children with autism," *International Conference on Social Robotics*, pp. 143–152, 2012.

[4] Aceituno-Valenzuela, L., Nuñez-Marcos, A., García-Zapirain, B., "Robotic coaches for physical exercise: A systematic review," *Journal of Ambient Intelligence and Humanized Computing*, vol. 11, no. 8, pp. 3277–3290, 2020.

[5] Speer, R., Chin, J., Havasi, C., "ConceptNet 5.5: An open multilingual graph of general knowledge," *AAAI Conference on Artificial Intelligence*, vol. 31, 2017.

[6] Fasolla, F. J., Mataric, M. J., "Socially assistive robotics," in *IEEE Handbook of Robotics*, pp. 1935–1972, 2016.

[7] Staubitz, F. M., Carty, C., Zeller, R., et al., "Effectiveness of robot-assisted gait training in individuals following stroke," *Physical Medicine and Rehabilitation*, vol. 45, no. 6, pp. 455–469, 2014.

[8] Lenat, D. B., Guha, R. V., "Building large knowledge-based systems," *Artificial Intelligence*, vol. 61, no. 1, pp. 3–39, 1993.

[9] Zhang, S., Roller, S., Goswami, A., et al., "Can retrieval-augmented language models explain their predictions?" *arXiv preprint arXiv:2210.12466*, 2022.

[10] D'Elia, D., Stafford, R. Q., Broadbent, E., et al., "Effects of robot expressiveness on emotion recognition," *International Conference on Social Robotics*, pp. 237–246, 2014.

[11] Salem, M., Eyssel, F., Rohlfing, K., et al., "To err is robot: How humans assess and act upon robot mistakes," *Social Robotics*, pp. 357–367, 2013.

[12] Ahn, M., Brohan, A., Brown, N., et al., "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[13] Sap, M., Gabriel, S., Qin, L., et al., "Social bias frames," *ACL*, 2020.

[14] Ye, J., Qiu, Z., Ye, K., et al., "A survey on large language models for machine learning: Conversational and prompt learning," *arXiv preprint*, 2023.

[15] Stojnic, G., Demartsev, V., Frey, B. S., et al., "Knowledge graphs for robotics planning," *International Symposium on Experimental Robotics*, pp. 134–145, 2021.

[16] Brooks, R. A., "The relationship between matter and life," *Nature*, vol. 409, no. 6818, pp. 409–411, 2001.

[17] Bekey, G. A., "Autonomous robots," *MIT Press*, 2005.

[18] Ultralytics, "YOLOv8: Real-time object detection," https://github.com/ultralytics/ultralytics, 2023.

[19] Wolf, T., Debut, L., Sanh, V., et al., "Hugging Face's transformers: State-of-the-art natural language processing," *arXiv preprint arXiv:1910.03771*, 2019.

[20] Serengil, S. I., Ozpinar, A., "DeepFace: Face recognition via deep learning," *IEEE International Symposium on Innovations in Intelligent Systems and Applications*, pp. 1–4, 2016.

[21] Cao, Z., Simon, T., Wei, S. E., Sheikh, Y., "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 172–186, 2019.

[22] Vosk Speech Recognition, https://github.com/alphacep/vosk-api, 2021.

[23] Google Cloud Speech-to-Text, https://cloud.google.com/speech-to-text, 2023.

[24] Brown, T., Mann, B., Ryder, N., et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[25] Lewis, P., Perez, E., Piktus, A., et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[26] Hogan, A., Blomqvist, E., Cochez, M., et al., "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[27] Sakakibara, S., Kawaguchi, R., Aoki, S., et al., "Designing robot personalities via behavior trees and language models," *HRI '23*, pp. 123–131, 2023.

[28] Thawani, A., Singh, A., Prabhumoye, S., et al., "Factual error correction for abstractive summarization models," *EMNLP*, pp. 7720–7732, 2021.

[29] Leite, I., LoFi, C., Dautenhahn, K., "Processing multimodal dialogue with the Multimodal Dialogue Manager," *International Symposium on Robotics*, 2015.

[30] Ijspeert, A. J., Nakanishi, J., Schaal, S., "Movement imitation with nonlinear dynamical systems in humanoid robots," *IEEE Transactions on Robotics*, vol. 18, no. 5, pp. 670–682, 2002.

[31] Sucan, I. A., Chitta, S., "MoveIt: An open source robotics manipulation platform," *IEEE Robotics & Automation Magazine*, vol. 20, no. 3, pp. 22–34, 2013.

[32] Mellinger, D., Kumar, V., "Minimum snap trajectory generation and control for quadrotors," *IEEE International Conference on Robotics and Automation*, pp. 2520–2525, 2011.

[33] Lynxmotion, "SES900 Dual Arm Robot Kit," https://www.robotshop.com/, 2023.

[34] Breazeal, C., "Toward sociable robots," *IEEE Robotics & Automation Magazine*, vol. 7,

no. 4, pp. 12–19, 2000.

[35] van den Oord, A., Sifre, K., Kalchbrenner, N., et al., "WaveNet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.

[36] Guiochet, J., Machin, M., Waeselynck, H., "Safety-critical advanced robots: a survey," *Robotics and Autonomous Systems*, vol. 94, pp. 43–52, 2017.

[37] OptiTrack, "Motion Capture Systems," https://optitrack.com/, 2023.

[38] Zita, C., Stehr, R., Halit, et al., "Real-time collision avoidance in humanoid robots," *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 110–127, 2008.

[39] Goertzel, B., "Artificial general intelligence: concept, state of the art, and future prospects," *Journal of Artificial General Intelligence*, vol. 5, no. 1, pp. 1–46, 2014.

[40] Agre, P. E., Chapman, D., "Pengi: An implementation of a theory of activity," *AAAI*, vol. 7, pp. 268–272, 1987.

[41] Schaal, S., "Learning from demonstration," *Advances in Neural Information Processing Systems*, pp. 1040–1046, 1999.